

OOP

programare orientată pe obiecte

CURS02: CLASE.

MODELUL DE MEMORIE.

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
Universitatea Politehnica Bucureşti

CE POATE CONTINE O CLASA?

alexandru.olteanu@upb.ro

```
class Point2D {
  int x;
  int y;

  void print() {
    cout<<"("<<this.x<<","<<this.y<<")"<<endl;
  }
}

// ... maybe in different file(s), far far away
Point2D p = new Point2D();
p.print();
```


alexandru.olteanu@upb.ro

```
class Point2D {
  int x;
  int y;

  void print() {
    System.out.println("("+this.x+","+this.y+")")
  }
}

// ... maybe in different file(s), far far away
Point2D p = new Point2D();
p.print();
```


alexandru.olteanu@upb.ro

- campuri / proprietati
- metode
  - constructori
  - accesori (setter si getter)
- specificatori de acces

alexandru.olteanu@upb.ro

SPECIFICATORI DE ACCES

alexandru.olteanu@upb.ro

- public - permite acces complet din exteriorul clasei curente
- private - limiteaza accesul doar în cadrul clasei curente
- protected - limiteaza accesul doar în cadrul clasei curente, al altor clase din acelasi pachet sau al descendentilor* ei
- (default) - în cazul în care nu este utilizat explicit nici unul din specificatorii de acces de mai sus, accesul este permis doar în cadrul pachetului (package private)

alexandru.olteanu@upb.ro

```
class Point2D {
  private int x;
  private int y;

  public void print() {
    System.out.println("("+this.x+","+this.y+")")
  }
}
```


alexandru.olteanu@upb.ro

```
class GeneratorNume {
  private String [] vocabular = {
      new String ("Ana") ,
      new String (" Mihai ") ,
      new String (" Robert ")
  };
  private Random randomGenerator = new Random () ;

  public String genNume () {
      return vocabular [ randomGenerator . nextInt (3) ];
  }
}
...
GeneratorNume gn = new GeneratorNume () ;
System.out.println(gn.genNume () ) ;
```


alexandru.olteanu@upb.ro

SETTER / GETTER (ACCESSOR)

alexandru.olteanu@upb.ro

```
class Point2D {
  private int x;
  private int y;

  public void setX(int x) {
    this.x = x;
  }

  public int getX() {
    return x;
  }
}
```


alexandru.olteanu@upb.ro

De ce sa folosim Setter/Getter?
Pot exista nivele de acces diferite pentru setter si getter (sau chiar unul sa nu existe)

```
public class Employee {
    private int salary ;
    public int getSalary () {
      return salary ;
    }
    protected void setSalary (int salary ) {
      this . salary = salary ;
    }
}
```


alexandru.olteanu@upb.ro

De ce sa folosim Setter/Getter?
Se poate ascunde reprezentarea intern :

```
public class Employee {
    private String street ;
    private int number ;
    private String city ;

    public int getAddress () {
        return street +", "+ number +", "+ city ;
    }
}
```


alexandru.olteanu@upb.ro

De ce sa folosim Setter/Getter?
Se pot face validari:

```
public class Employee {
  private String email ;
  public void setEmail ( String email ) {
    if (! EmailChecker.isValid ( email ) ) {
      System.out.println ( email ) ;
    } else {
      this.email = email ;
    }
  }
}
```


alexandru.olteanu@upb.ro

De ce sa folosim Setter/Getter?
Se pot face conversii:

```
public class Employee {
    private float height ;
    public void setHeight ( float height , String measure ) {
        if (! measure . equals ("m") ) {
          this.height = height / 0.3048;
        } else {
          this.height = height ;
        }
    }
}
```


alexandru.olteanu@upb.ro

Boilerplate code

In C#:

```
public abstract class Foo {
    public virtual string Hello { get; set; }
}
```

In Java: Project Lombok

alexandru.olteanu@upb.ro

CONSTRUCTORI

alexandru.olteanu@upb.ro

Default constructor (no-arg constructor): provides the default values to the object like 0, null etc. depending on the type

```
public class Magazin {
    private String brand ;
    public Magazin () {
        brand = "de inchiriat ";
    }
}
...
Magazin item = new Magazin () ;
```


alexandru.olteanu@upb.ro

Parameterized constructor: provide dierent values to the distinct objects

```
public class Magazin {
  private String brand ;
    public Magazin ( String brand ) {
      this.brand = brand ;
    }
}
...
Magazin item = new Magazin (" IKEA ") ;
```


alexandru.olteanu@upb.ro

A class can have any number of constructors that dier in parameter lists. The compiler differentiates these constructors by taking into account the number of parameters in the list and their type

```
public class Magazin {
  private String brand ;
  private Integer suprafata ;
  public Magazin ( Integer suprafata ) {
    this.suprafata = suprafata ;
    this.brand = "de inchiriat ";
  }
  public Magazin ( Integer suprafata , String brand ) {
    this.suprafata = suprafata ;
    this.brand = brand ;
  }
}
```

In Java, there is no Copy Constructor per-se, but there are many ways to copy the values of one object into another:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

```
public class Training {
  private int pushups ;
  private int crunches ;
  public Training ( Training otherTraining ) {
    this . pushups = otherTraining . pushups ;
    this . crunches = otherTraining . crunches ;
  }
}
```

In C++, se apeleaza la iesirea din scope a variabilei:

```
class Catalog {
  private :
    int * note ;
  public :
    Catalog (int count = 20) {
      note = new int [ count ];
    }
    ~Catalog () {
      delete [] note ;
    }
};
int main () {
Catalog * grupa321CD = new Catalog (30) ; // -> constructor
...
delete grupa321CD ;                       // -> destructor
}
```

In C++, se apeleaza la iesirea din scope a variabilei:

```
class Catalog {
  private :
    int * note ;
  public :
    Catalog (int count = 20) {
      note = new int [ count ];
    }
    ~Catalog () {
      delete [] note ;
    }
};
int main () {
Catalog * grupa321CD = new Catalog (30) ; // -> constructor
...
//delete grupa321CD ;
}                                          // -> destructor
```


alexandru.olteanu@upb.ro

In Java exista functia finalize, dar cu adevarat nu stim cand este apelata pentru ca de gestiunea memoriei se ocupa Garbage Collector

Java Garbage Collection Basics (Oracle)

alexandru.olteanu@upb.ro

MODELUL DE MEMORIE

alexandru.olteanu@upb.ro

Pointeri vs Referinte in C/C++

```
int x = 13 , y = 14;
int *px = &x, &rx = x;
int *px , &rx; // eroare : referintele sunt initializate la crea
px = &y; rx = y; // nu se modifica referinta , ci variabila refe
px ++; rx++; // nu se modifica referinta , ci variabila referita
```


alexandru.olteanu@upb.ro

Pointeri vs Referinte in C/C++

```
void swap (int first, int second) {
    int temp = first; first = second; second = temp;
}

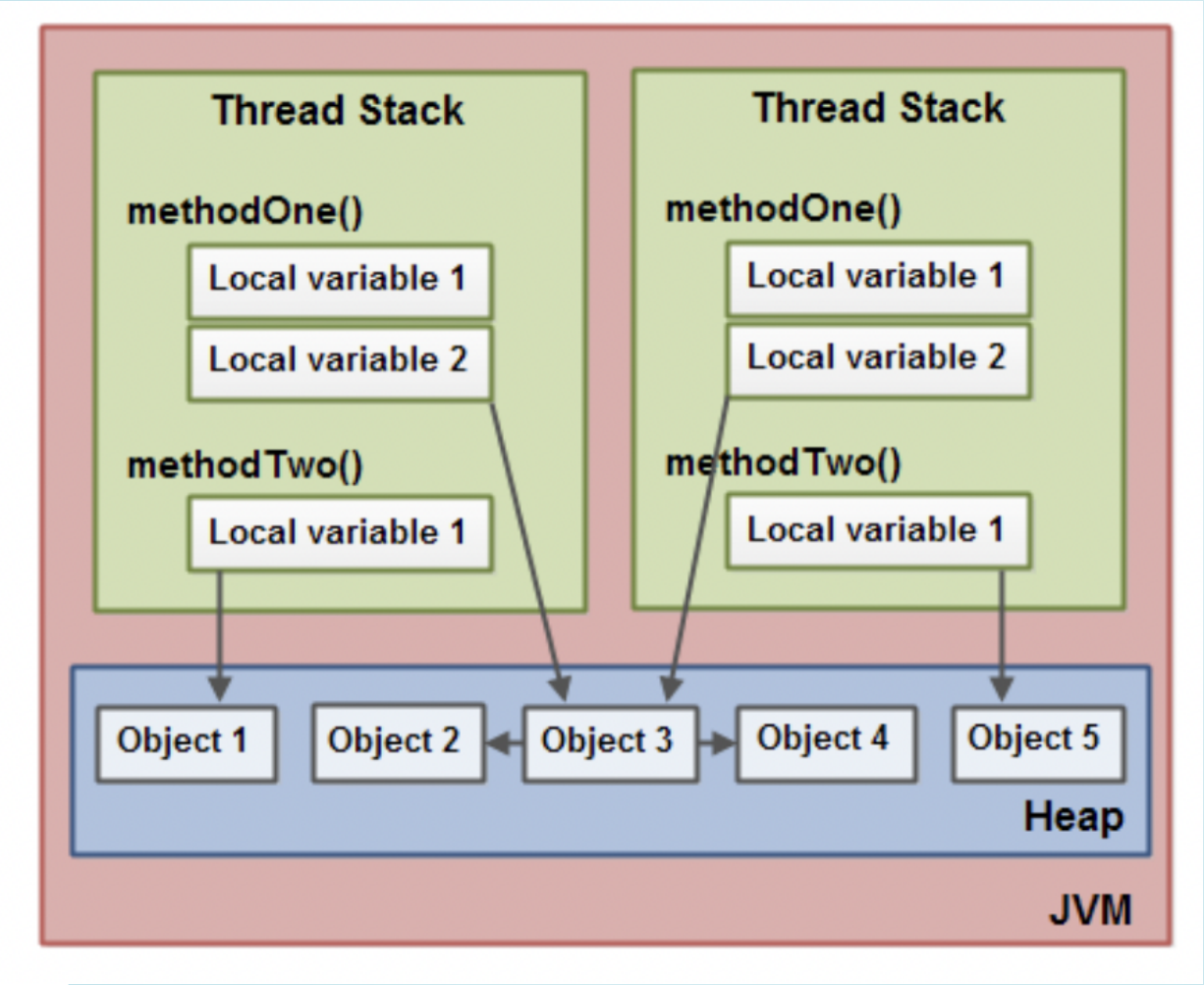
void swap (int* first, int* second) {
    int* temp = first; first = second; second = temp;
}

void swap (int* first, int* second) {
    int* temp; *temp = *first; *first = *second; *second = *tem
}

void swap (int& first, int& second) {          // doar in C++
    int temp = first; first = second; second = temp;
}
```


alexandru.olteanu@upb.ro

In Java avem doar referinte, nu avem pointeri



alexandru.olteanu@upb.ro

În Java, variabile din tipuri de date primitive pot fi locale unei metode sau membrii unui obiect

```
public class Album {
  public static void main ( String [] args ) {
    int price ;
  }
}
```

```
public class Album {
  int price ;
  public static void main ( String [] args ) {
  }
}
```

variabilele locale nu sunt initializate

```
public class Album {
  public static void main ( String [] args ) {
    int price ;
    price ++; // eroare de compilare
  }
}
```

membrii claselor sunt initializati cu 0 sau null, în functie de tip

```
public class Album {
  int price ;
  public static void main ( String [] args ) {
    price ++; // no error, bad programming style
  }
}
```

Pentru referintele la clase, la declararea fara initializare se creeaz o referinta nula :

```
Magazin spatiuDeInchiriat;
```

La initializare, se aloca spatiu pe heap si se apeleaz un constructor:

```
Magazin magazinSport = new Magazin (" Articole Sportive ") ;
```


alexandru.olteanu@upb.ro

Doua referinte la acelasi obiect: se modica obiectul

```
public Arena {
  public int seats ;
  Arena (int seats ) {
    this . seats = seats ;
  }
}

Arena arenaNationala = new Arena (55000) ;
stadionulNational = arenaNationala ;
stadionulNational.seats += 600;
System.out.println ( arenaNationala . seats ) ; // 55000 ? 55600
```


alexandru.olteanu@upb.ro

- == tests for reference equality (whether they are the same object)
- .equals() tests for value equality (whether they are logically "equal")

```
String name1 = new String (" Michael ");
String name2 = name1 ;
String name3 = new String (" Michael ") ;

System .out. println ( name1 == name2 ) ;
System .out. println ( name1 == name3 ) ;
System .out. println ( name1 . equals ( name3 ) ) ;
```


alexandru.olteanu@upb.ro

Sigur ati inteles?

```
public class MyProgram {
  public static void main ( String args [])
  {
    Integer a = Integer . valueOf (1) ;
    Integer b = Integer . valueOf (1) ;
    System .out. println (a==b) ;
    Integer x = Integer . valueOf (10001) ;
    Integer y = Integer . valueOf (10001) ;
    System .out. println (x==y) ;
  }
}
```


alexandru.olteanu@upb.ro

Transferul parametrilor la apelul functiilor este crucial pentru o functionare corecta:

- variabile de tip primitiv se transfera prin copiere pe stiva
  - orice modicare din functie a valorii variabilei NU VA FI VIZIBILA
- obiecte
  - se transfera prin referinta pe stiva
  - orice modicare din functie a referintei (e.g. p = new Player()) NU VA FI VIZIBILA

Reading material

- Lab01: Java Basics
- Lab02: Constructori si Referinte
- Memory Model
- Rule of Three/Five/Zero

alexandru.olteanu@upb.ro